

A3-PS022-TC164

eBPF/XDP Rate Limiting for DNS Query Floods in a Lab

Team name : Sparrow_IITG

t.aryashi@iitg.ac.in

j.toshit@iitg.ac.in

AIORI Hackathon 3

October 14, 2026

Problem Description

1. **Server Saturation via UDP Floods:** High-volume UDP DNS query floods saturate Linux servers because the operating system kernel spends substantial CPU resources allocating socket buffers before packets reach user space.
2. **Pre-Allocation Vulnerability:** Without driver-level filtering, malicious packets force the system to waste resources on memory allocation before they can be dropped.
3. **Degraded Legitimate Query Performance:** High-volume query floods cause severe spikes in CPU utilization and reduce legitimate query success rates.
4. **Lack of Low-Level Rate Limiting:** Standard network configurations lack efficient mechanisms to enforce both per-source and global rate limits directly inside the network interface driver.

Background

High-volume UDP-based DNS query floods can overwhelm conventional Linux servers, as significant CPU resources are consumed by kernel-level packet processing and socket buffer allocation before packets reach user space. Traditional filtering mechanisms, such as *nftables*, provide a useful performance baseline but operate later in the network processing pipeline. This project leverages the ***eXpress Data Path (XDP)*** to execute eBPF programs directly within the network interface driver, enabling packets to be filtered or dropped before expensive kernel processing and memory allocation occur. By implementing bounded UDP/DNS packet parsing alongside per-source and global rate-limiting mechanisms, the proposed approach aims to mitigate DNS query floods efficiently, reduce CPU utilization, and maintain high success rates for legitimate DNS queries.

Literature & RFC Review

- **XDP / eBPF [1]** Høiland-Jørgensen et al. show XDP runs in the driver before skb allocation, in native or generic mode. Our hook point, and the reason a generic-mode fallback is documented.
- **Production precedent [2]** Cloudflare's L4Drop compiles DDoS rules to XDP and drops attack traffic at the driver. We study a narrower, reproducible case: DNS-specific bounded parsing plus rate limits, benchmarked against nftables.
- **DNS RRL [3]** Rate-limits responses inside the DNS server (application layer). We rate-limit queries earlier, in the driver; the two are complementary, not competing.
- **DNS wire format [4]** RFC 1035: a DNS message starts with a fixed 12-byte header. This is what makes header-only, bounded parsing possible without reading the question name.
- **Spoofing & reflection [5,6]** BCP 38 / RFC 5358: spoofed sources enable reflection and are fixed by ingress filtering, not by a per-source limit. We state this limit explicitly rather than claim to eliminate reflection.
- **Rate metering [7]** Token-bucket metering (RFC 2697) allows short legitimate bursts while bounding the sustained rate; nftables meters are the baseline equivalent [8].

Proposed Solution

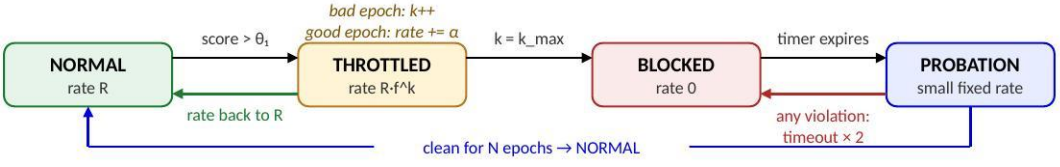
XDP program on the DNS server's NIC driver: PASS or DROP before any skb is allocated

- **Bounded parsing:** every read is checked against data_end: Ethernet → IPv4 (honours IHL) → UDP port 53 → 12-byte DNS header (QR = 0, QDCOUNT ≥ 1); no name or semantic parsing. Non-DNS passes untouched; malformed port-53 packets are dropped and counted.
- **Per-source limit:** lazily refilled token bucket (ktime, fixed-point, no timers) per source IPv4 in a size-capped LRU hash [9]; bounded memory under source churn (eviction resets penalty state).
- **Adaptive layer:** per-source state machine NORMAL → THROTTLED → BLOCKED → PROBATION (slide 6); EWMA of the per-epoch drop ratio drives transitions with hysteresis; multiplicative decrease, additive recovery, capped block backoff, never a permanent block.
- **Global limit:** token bucket for all port-53 queries, checked after the per-source limit; two tiers (reserved for established sources, tighter pool for new or unknown ones) so spoofed multi-source floods cannot starve legit clients.
- **Tunable and observable:** rates, burst, thresholds and epoch in a config map set from user space (no reload); per-CPU pass / drop-by-reason counters; bpf_spin_lock on per-source entries.
- **Evaluation:** Host A (legit client + flood generator) → Host B (DNS server): 3 patterns × 4 configs (no filter, nftables, XDP static, XDP adaptive) at identical thresholds; throughput, CPU, drops, legit success.
- **Scope:** isolated lab only; no public flood testing; no claim of eliminating reflection or detecting PRSD.

Optimization Proposed

Design choice	Naive approach	Proposed optimization	Expected effect
Parsing & state	Whole DNS message; unbounded hash map	Bounded L2–L4 + 12-byte header; size-capped LRU hash	Fixed per-packet cost; bounded memory under churn
Rate algorithm	Fixed-window counter	Lazy token bucket / GCRA: ktime, fixed-point, no timers	Tolerates bursts; no window-edge spikes
Global budget	One first-come bucket; shared lock	Two-tier (established vs new) + per-CPU sub-budgets	Legit clients survive spoofed floods; low contention
Verdict & telemetry	Binary or permanent block; shared counters	Adaptive state machine (below); per-CPU counters, sampled events	No permanent lockout; no hot-path contention

Adaptive per-source state machine (AIMD + circuit breaker)



score = EWMA of the per-epoch drop ratio; hysteresis: enter THROTTLED at θ_1, leave at $\theta_0 < \theta_1$; epochs evaluated lazily per packet. Before/after measurements will be added after benchmarking; no results are claimed at proposal stage. Token bucket: RFC 2697 [7]; AIMD [10]; circuit breaker [13].

Solution Architecture & Design

Two-host lab: generator → XDP-protected DNS server, with a user-space controller

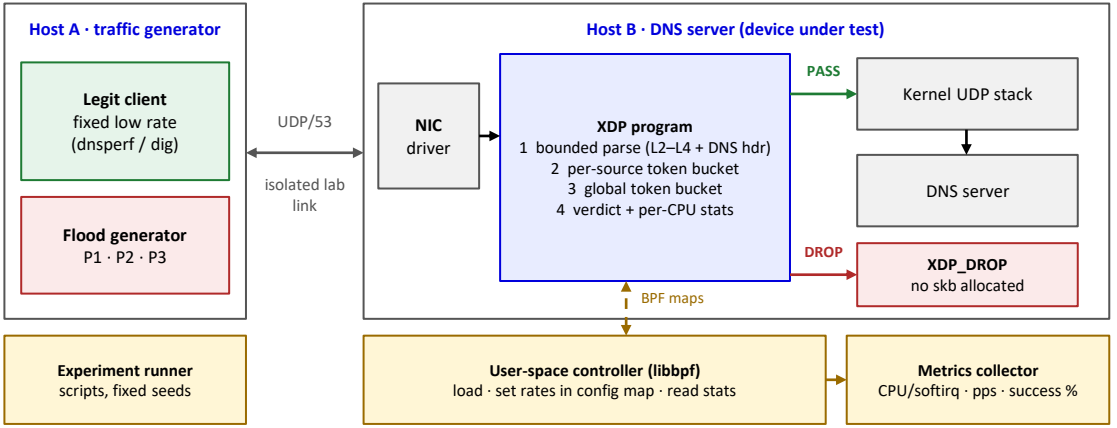


Figure: System architecture (component view)

Data Flow Diagram

Path of one packet through the XDP program, and the data stores it touches

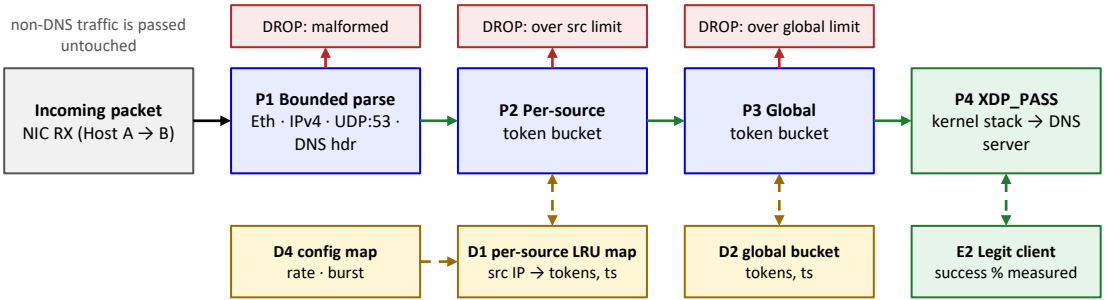


Figure: Data flow diagram (DFD) of the packet-processing path

Deployment Timeline

Milestones

- **M1 Lab and baseline:** two hosts (veth/netns or VMs); record kernel, NIC, driver and XDP mode (native / generic); no-filter run.
- **M2 Traffic harness:** scripted legitimate client and 3 flood patterns; isolated network only.
- **M3 nftables baseline:** per-source and global limits at the same thresholds.
- **M4 XDP filter:** bounded parser, two buckets, config and stats maps; tests with crafted and malformed packets.
- **M5 Benchmark:** 3 patterns × 3 configs × ≥ 5 runs; collect CPU, pps, drops, legit success.
- **M6 Analysis and demo:** report, graphs, live demo with pseudocode.

Methodology: identical conditions

- **P1 single-source:** one address, constant high rate (tests the per-source limit).
- **P2 distributed:** many addresses, each under the per-source limit (tests the global limit).
- **P3 bursty + legit load:** on/off flood bursts while the legit client runs (tests recovery and collateral damage).
- **Configs:** A no filter · B nftables · C XDP, same thresholds.
- **Metrics:** throughput (pps), CPU % incl. softirq, drops, legit-query success %.
- **Rigor:** same hardware, kernel, duration and traffic; ≥ 5 runs, report mean and spread.

Bibliography I

1 T. Høiland-Jørgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern, D. Miller. "The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel." In: Proc. ACM CoNEXT, 2018.

2 A. Fabre. "L4Drop: XDP DDoS Mitigations." Cloudflare Blog, 28 Nov 2018. blog.cloudflare.com/l4drop-xdp-ebpf-based-ddos-mitigations

3 P. Vixie, V. Schryver. "DNS Response Rate Limiting (DNS RRL)." ISC-TN-2012-1-Draft1, Apr 2012. isc.org/pubs/tn/isc-tn-2012-1.txt

4 P. Mockapetris. "Domain Names: Implementation and Specification." RFC 1035, 1987.

5 P. Ferguson, D. Senie. "Network Ingress Filtering: Defeating Denial of Service Attacks which employ IP Source Address Spoofing." RFC 2827 (BCP 38), 2000.

6 J. Damas, F. Neves. "Preventing Use of Recursive Nameservers in Reflector Attacks." RFC 5358 (BCP 140), 2008.

7 J. Heinanen, R. Guérin. "A Single Rate Three Color Marker." RFC 2697, 1999.

8 nftables wiki: rate limiting with meters and dynamic sets. wiki.nftables.org

9 Linux kernel documentation: BPF (maps, ring buffer). docs.kernel.org/bpf

Thank You for Your Attention!